

Importing Libraries and data set

```
# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like
competitions, datasets, and models to your session
# Learn more about kagglehub:
https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')

/kaggle/input/datasets/sabirbagwan/engineering-students-performance-
dataset/synthetic_student_dataset.csv

df = pd.read_csv("/kaggle/input/datasets/sabirbagwan/engineering-
students-performance-dataset/synthetic_student_dataset.csv")
```

Let's view our data

```
df.head()
```

Name	Gender	Hours_of_Study	Hours_of_Play	Marks_10th	Marks_12th
\					

0	Student_1	Male	7.77	4.34	58
57					
1	Student_2	Female	5.58	2.89	58
62					
2	Student_3	Male	4.24	2.19	53
91					
3	Student_4	Male	9.06	3.63	84
57					
4	Student_5	Male	1.11	2.43	66
79					

	CET_Score	Sem1_CGPA	Sem2_CGPA	Sem3_CGPA	Sem4_CGPA	Sem5_CGPA	\
0	112	6.94	5.18	9.49	6.52	8.12	
1	71	6.20	7.37	9.54	6.96	6.41	
2	119	5.47	6.68	5.44	5.90	6.89	
3	174	6.57	9.60	9.68	7.89	5.51	
4	198	5.48	5.06	9.13	9.79	8.60	

	Sem6_CGPA	Sem7_CGPA	Sem8_CGPA	Avg_CGPA	Placed	Package
0	7.79	6.70	6.84	7.19750	No	0.00
1	8.54	8.10	7.12	7.53000	No	0.00
2	5.89	9.45	6.80	6.56500	No	0.00
3	6.26	7.48	8.99	7.74750	Yes	8.94
4	9.05	9.12	7.62	7.98125	Yes	18.67

df.shape *#checking shape of dataset*

(900, 19)

df.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 900 entries, 0 to 899

Data columns (total 18 columns):

#	Column	Non-Null Count	Dtype
0	Name	900 non-null	object
1	Gender	900 non-null	object
2	Hours_of_Study	900 non-null	float64
3	Hours_of_Play	900 non-null	float64
4	Marks_10th	900 non-null	int64
5	Marks_12th	900 non-null	int64
6	CET_Score	900 non-null	int64
7	Sem1_CGPA	900 non-null	float64
8	Sem2_CGPA	900 non-null	float64
9	Sem3_CGPA	900 non-null	float64
10	Sem4_CGPA	900 non-null	float64
11	Sem5_CGPA	900 non-null	float64
12	Sem6_CGPA	900 non-null	float64
13	Sem7_CGPA	900 non-null	float64

```

14 Sem8_CGPA      900 non-null    float64
15 Avg_CGPA       900 non-null    float64
16 Placed         900 non-null    object
17 Package        900 non-null    float64
dtypes: float64(12), int64(3), object(3)
memory usage: 126.7+ KB

df.isnull().sum() #checking total number of null values in dataset
Name      0
Gender    0
Hours_of_Study  0
Hours_of_Play  0
Marks_10th  0
Marks_12th  0
CET_Score  0
Sem1_CGPA  0
Sem2_CGPA  0
Sem3_CGPA  0
Sem4_CGPA  0
Sem5_CGPA  0
Sem6_CGPA  0
Sem7_CGPA  0
Sem8_CGPA  0
Avg_CGPA   0
Placed     0
Package    0
Academic_Score  0
dtype: int64

df.dropna(inplace=True)

df.columns

Index(['Name', 'Gender', 'Hours_of_Study', 'Hours_of_Play',
      'Marks_10th',
      'Marks_12th', 'CET_Score', 'Sem1_CGPA', 'Sem2_CGPA',
      'Sem3_CGPA',
      'Sem4_CGPA', 'Sem5_CGPA', 'Sem6_CGPA', 'Sem7_CGPA',
      'Sem8_CGPA',
      'Avg_CGPA', 'Placed', 'Package'],
      dtype='object')

```

Label Encoding

```

from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

```

```
df["Gender"] = le.fit_transform(df["Gender"]) # Male = 1, Female = 0
df["Placed"] = le.fit_transform(df["Placed"]) # Yes = 1, No = 0
```

```
df.head()
```

	Name	Gender	Hours_of_Study	Hours_of_Play	Marks_10th
0	Student_1	1	7.77	4.34	58
1	Student_2	0	5.58	2.89	58
2	Student_3	1	4.24	2.19	53
3	Student_4	1	9.06	3.63	84
4	Student_5	1	1.11	2.43	66

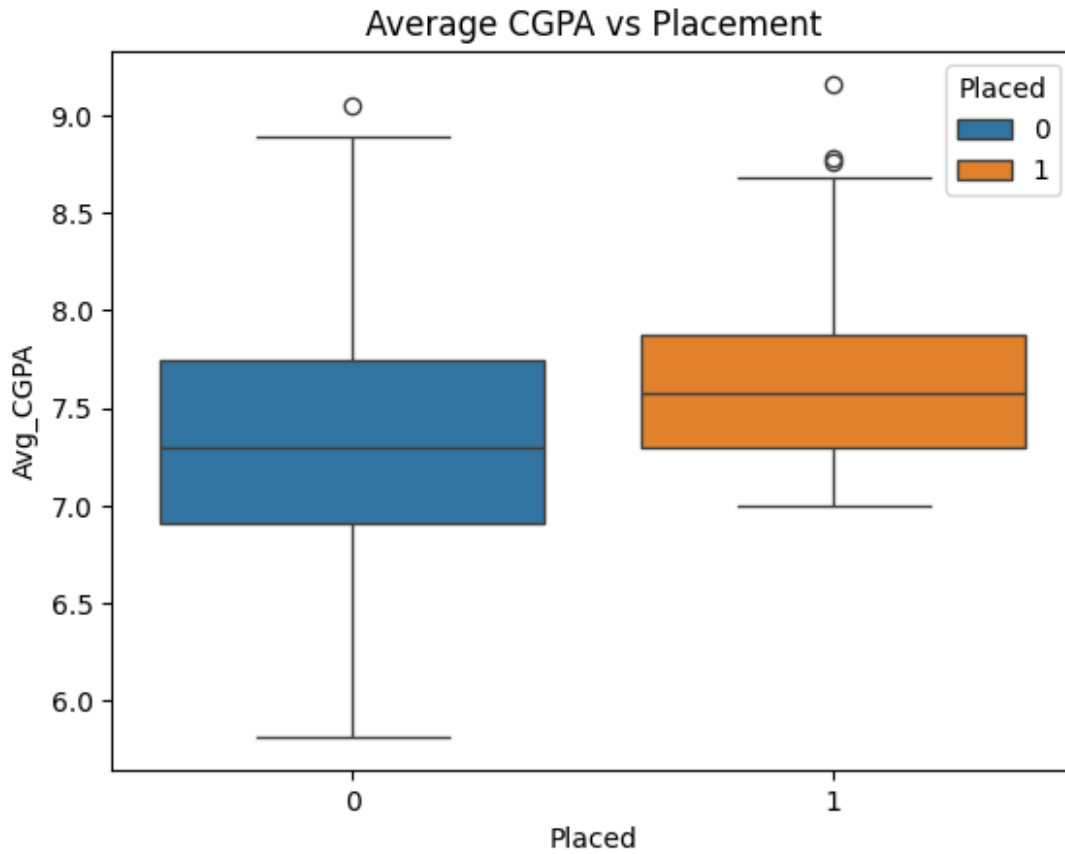
	CET_Score	Sem1_CGPA	Sem2_CGPA	Sem3_CGPA	Sem4_CGPA	Sem5_CGPA
0	112	6.94	5.18	9.49	6.52	8.12
1	71	6.20	7.37	9.54	6.96	6.41
2	119	5.47	6.68	5.44	5.90	6.89
3	174	6.57	9.60	9.68	7.89	5.51
4	198	5.48	5.06	9.13	9.79	8.60

	Sem6_CGPA	Sem7_CGPA	Sem8_CGPA	Avg_CGPA	Placed	Package
0	7.79	6.70	6.84	7.19750	0	0.00
1	8.54	8.10	7.12	7.53000	0	0.00
2	5.89	9.45	6.80	6.56500	0	0.00
3	6.26	7.48	8.99	7.74750	1	8.94
4	9.05	9.12	7.62	7.98125	1	18.67

Box plot to Average CGPA and Placement:

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(x="Placed", y="Avg_CGPA", data=df, hue="Placed")
plt.title("Average CGPA vs Placement")
plt.show()
```



Feature Enginnering

```
df.columns
```

```
Index(['Name', 'Gender', 'Hours_of_Study', 'Hours_of_Play',
      'Marks_10th', 'Marks_12th', 'CET_Score', 'Sem1_CGPA', 'Sem2_CGPA',
      'Sem3_CGPA', 'Sem4_CGPA', 'Sem5_CGPA', 'Sem6_CGPA', 'Sem7_CGPA',
      'Sem8_CGPA', 'Avg_CGPA', 'Placed', 'Package'],
      dtype='object')
```

```
#creating a new column to understand rowise calulation
```

```
df["Academic_Score"] =
df[["Marks_10th", "Marks_12th", "CET_Score"]].mean(axis=1) # Rows wise Cal
```

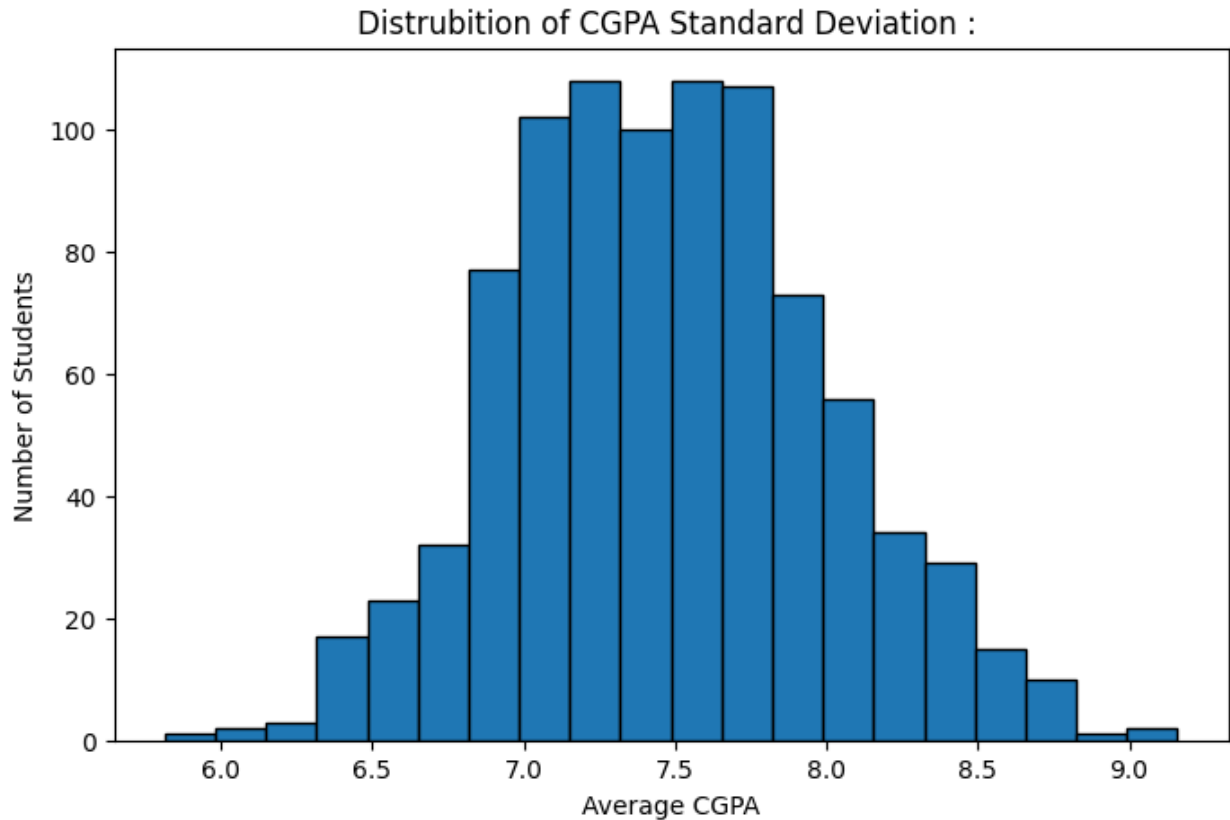
```
df.head()
```

```
      Name  Gender  Hours_of_Study  Hours_of_Play  Marks_10th
Marks_12th  \
```

0	Student_1	1	7.77	4.34	58		
57							
1	Student_2	0	5.58	2.89	58		
62							
2	Student_3	1	4.24	2.19	53		
91							
3	Student_4	1	9.06	3.63	84		
57							
4	Student_5	1	1.11	2.43	66		
79							
	CET_Score	Sem1_CGPA	Sem2_CGPA	Sem3_CGPA	Sem4_CGPA	Sem5_CGPA	\
0	112	6.94	5.18	9.49	6.52	8.12	
1	71	6.20	7.37	9.54	6.96	6.41	
2	119	5.47	6.68	5.44	5.90	6.89	
3	174	6.57	9.60	9.68	7.89	5.51	
4	198	5.48	5.06	9.13	9.79	8.60	
	Sem6_CGPA	Sem7_CGPA	Sem8_CGPA	Avg_CGPA	Placed	Package	
	Academic_Score						
0	7.79	6.70	6.84	7.19750	0	0.00	
75.666667							
1	8.54	8.10	7.12	7.53000	0	0.00	
63.666667							
2	5.89	9.45	6.80	6.56500	0	0.00	
87.666667							
3	6.26	7.48	8.99	7.74750	1	8.94	
105.000000							
4	9.05	9.12	7.62	7.98125	1	18.67	
114.333333							

Distrubition of CGPA Standard Deviation

```
plt.figure(figsize=(8,5))
plt.hist(df["Avg_CGPA"],bins=20,edgecolor="black")
plt.xlabel("Average CGPA")
plt.ylabel("Number of Students")
plt.title("Distrubition of CGPA Standard Deviation : ")
plt.show()
```



Let's compare Regressor models

Splitting data set into train and test

```
X=df.drop(["Name", "Placed", "Package"], axis=1)
y=df["Package"]
print(f"X Shape : {X.shape}")
print(f"Y Shape : {y.shape}")

X Shape : (900, 16)
Y Shape : (900,)

from sklearn.model_selection import train_test_split
X_train, X_test, y_train,
y_test=train_test_split(X,y,test_size=0.2,random_state=42)
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

X_train shape: (720, 16)
X_test shape: (180, 16)
```

```
y_train shape: (720,)
y_test shape: (180,)
```

Lets train our data on different Regression Models and then compare them using Metrics

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

lr = LinearRegression()
print(f"Training on Linear Regression: {lr.fit(X_train,y_train)}")

rfr = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
print(f"Training on RandomForest Regression:
{rfr.fit(X_train,y_train)}")

dtr = DecisionTreeRegressor(
    random_state=42
)
print(f"Training on DecisionTree Regression:
{dtr.fit(X_train,y_train)}")

Training on Linear Regression: LinearRegression()
Training on RandomForest Regression:
RandomForestRegressor(random_state=42)
Training on DecisionTree Regression:
DecisionTreeRegressor(random_state=42)
```

Lets Compare the Metrics and Scores each algo achieved during Training:

```
from sklearn.metrics import (
    r2_score,
    mean_absolute_error,
    mean_squared_error,
    root_mean_squared_error
)

import numpy as np
```

```
def get_regression_metrics(model,X_test,y_test,name):
    y_pred = model.predict(X_test)

    return {
        "Model":name,
        "R2_Score":r2_score(y_test,y_pred),
        "Mean Absolute Error":mean_absolute_error(y_test,y_pred),
        "Mean Squared Error":mean_squared_error(y_test,y_pred),
        "Root Mean Squared
Error":root_mean_squared_error(y_test,y_pred)
    }

metrics = [
    get_regression_metrics(lr,X_test,y_test,"Linear Regression"),
    get_regression_metrics(rfr,X_test,y_test,"RandomForest
Regression"),
    get_regression_metrics(dtr,X_test,y_test,"DecisionTree
Regression")
]
```

```
df_regression_metrics =
pd.DataFrame(metrics).set_index("Model").round(4)
```

```
df_regression_metrics
```

	R2_Score	Mean Absolute Error	Mean Squared Error
Model \			
Linear Regression	0.3912	4.0209	26.2649
RandomForest Regression	0.6907	2.2526	13.3435
DecisionTree Regression	0.4586	2.6653	23.3571
	Root Mean Squared Error		
Model			
Linear Regression		5.1249	
RandomForest Regression		3.6529	
DecisionTree Regression		4.8329	